Skip to main content

ILLINOIS

Search in course

Search

- English
-
- Menu pop-up Collapsed

B

Hide menu

- **Week 4 Information**
- **MapReduce and Hadoop**
- **HDFS**
- **Apache Spark**
- **[Optional machine problem]**
- **Week 4 Graded Activities**
  -

Reading: ReadingTutorial: Docker installation

. Duration: 10 minutes10 min

  -

Graded Assignment: Week 4 Quiz

Submitted

○

Programming Assignment: Machine Problem 5: Spark MapReduce

. Duration: 6 hours6h

1. CS 498: Cloud Computing Applications
2. Week 4
3. Machine Problem 5: Spark MapReduce

PreviousNext

Programming Assignment: Machine Problem 5: Spark MapReduce

Graded • 5/5 points

Deadline

Pass this assignment by Mar 9, 11:59 PM CDT

InstructionsMy submissionsDiscussions

Instructions

1. Overview

Welcome to the Spark MapReduce programming assignment. You will implement the solution to this machine problem in Python. To work on this assignment, you need Docker Desktop installed.

2. General Requirements

Please note that our grader runs on a docker container **NOT connected to the internet**. Therefore, **no additional libraries are allowed** for this assignment **(you can only use the default libraries of Python but no pip installs)**. Also, you will **NOT be allowed to create any file or folder outside the current folder** (i.e., you can only create files and folders in the folder that your solutions are in).

3. Setup

Download the docker file, build a docker image and run it in a container. If you have already created this container, do not create a new one.

2

3

4

5

6

7

8

9

10

11

12

13

14

```
# clone the repository and find the docker file
git clone https://github.com/UIUC-CS498-Cloud/MP5_SparkMapReduce_Template.git
cd MP5_SparkMapReduce_Template/Docker

# build an image for mp5 based on the docker file
docker build -t mp5 -f Dockerfile .

# If you are a mac user instead of above command use
# docker build -t mp5 -f DockerfileMac .

# create a container named 'mp5-cntr' for mp5 using the image mp5
docker run -v <PATH_TO_LOCAL_REPO>:/MP5_SparkMapReduce_Template --name mp5-cntr -it mp5
```

4. Sorting

When selecting the top N items in a list, sorting is necessary. Use the following steps to sort:

1. Sort the list ASCENDING based on count first, then on the key. If the key is a string, sort lexicographically.

2. Select the bottom N items in the sorted list as Top items.

For example, to select top 5 items in the list {"A": 100, "B": 99, "C":98, "D": 97, "E": 96, "F": 96, "G":90}, first sort the items ASCENDING:

"G":90

"E": 96

"F": 96

"D": 97

"C":98

"B": 99

"A": 100

Then, the bottom 5 items are **A, B, C, D, F**.

Another example, to select 5 top items in the list {"43": 100, "12": 99, "44":98, "12": 97, "1": 96, "100": 96, "99":90}

"99":90

"1": 96

"100": 96

"12": 97

"44":98

"12": 99

"43": 100

Then, the bottom 5 items are **43, 12, 44, 12, 100**.

Submission

1. Requirements

This assignment will be graded based on **Python 3.6.**

2. Procedures

Step 1: Launch and go into the 'mp5-cntr' container after the setup. Note, because of the volume mount used in the setup the files on the docker image will reflect those on your local machine, and edits made locally will be appear inside of your container.

1

cd MP5_SparkMapReduce_Template/PythonTemplate

Step 2: Finish the exercises by editing the provided templates files. All you need to do is complete the parts marked with **TODO**. Please note that you are **NOT allowed to import any additional libraries.**

- Each exercise has one or more code templates. Simply edit these files.
- Our autograder runs the code on the provided Docker image.

More information about these exercises is provided in the next section.

Step 3: After you are done with the assignments, put all your 5 python files (TitleCountSpark.py, TopTitleStatisticsSpark.py, OrphanPagesSpark.py, TopPopularLinksSpark.py, PopularityLeagueSpark.py) into a .zip file named "**MP5.zip**". Remember not to include the parent folder (A common way to do this is: select all the relevant python files, right click and hit compress) Submit your "**MP5.zip**".

Exercise A: Top Titles

In this exercise, you will implement a counter for words in Wikipedia titles and find the top words used in these titles. We have provided a template for this exercise in the following file: **TitleCountSpark.py**

You need to make the necessary changes to parts marked with **TODO**.

Your application takes a list of Wikipedia titles (one in each line) as an input and first tokenizes them using the provided delimiters. It then makes the tokens lowercased and removes common words from the provided stopwords. Next, your application selects the

top **10** words, and finally, saves the count for them in the output. **Use the method in the Sorting section to select top words.**

You can test your output with:

1

spark-submit TitleCountSpark.py stopwords.txt delimiters.txt dataset/titles/ partA

In case you face the error below:

```
root@7faf018ba910:/MP5_SparkMapReduce_Template/PythonTemplate# spark-submit TitleCountSpark.py stopwords.txt delimiters.txt dataset/titles/ partA
Picked up _JAVA_OPTIONS:
#
# A fatal error has been detected by the Java Runtime Environment:
#
#  SIGILL (0x4) at pc=0x0000ffff9bd3fc5c, pid=16, tid=17
#
# JRE version:  (21.0.5+11) (build )
# Java VM: OpenJDK 64-Bit Server VM (21.0.5+11-Ubuntu-1ubuntu122.04, mixed mode, sharing, tiered, compressed oops, compressed class ptrs, g1 gc, linux-aarch64)
# Problematic frame:
# j  java.lang.System.registerNatives()V+0 java.base@21.0.5
#
```

Delete your existing container (not Image) and run a new one using the command below:

1

docker run -v <PATH_TO_LOCAL_REPO>:/MP5_SparkMapReduce_Template -
e JAVA_TOOL_OPTIONS="-XX:UseSVE=0" --name mp5-cntr -it mp5

If you want to check your output, run:

1

cat partA

Here is an example output showing the top **5** words in alphabetical order. Note that the autograder requires the top 10 (after they are chosen based on count):

```
county  1020
de      1684
list    1948
new     1077
school  1065
```

The order of lines matters. Please sort the output in alphabetic order as shown above. Also, make sure the key and value pairs in the final output are tab-separated.

Exercise B: Top Title Statistics

In this exercise, you will implement an application to find some statistics about the top words used in Wikipedia titles. We have provided a template for this exercise in the following file: **TopTitleStatisticsSpark.py**

You need to make the necessary changes to parts marked with **TODO**.

Your output from Exercise A will be used as input here. The application saves the following statistics about the top words in the output: “Mean” , “Sum”, “Minimum”, “Maximum”, and “Variance” of the counts. All values should be **floored** to be an integer. For the sake of simplicity, use integers in all calculations.

The following is the sample command we will use to run the application:

```
1

# spark-submit TopTitleStatisticsSpark.py partA partB
```

If you want to check your output, run:

```
1

# cat partB
```

Here is the output of an application that selects the top **5** words, though we still require the top 10 as described above:

```
Mean    1358
Sum     6794
Min     1020
Max     1948
Var     146686
```

Make sure the stats and the corresponding results are tab-separated.

Exercise C: Orphan Pages

In this exercise, you will implement an application to find orphan pages in Wikipedia. We have provided a template for this exercise in the following files: **OrphanPagesSpark.py**

You need to make the necessary changes to parts marked with **TODO**.

Your application takes a list of Wikipedia links (not Wikipedia titles anymore) as an input. All pages are represented by their ID numbers. Each line starts with a page ID, followed by a list of pages that the ID links to. The following is a sample line in the input:

```
2: 3 747213 1664968 1691047 4095634 5535664
```

In this sample, page 2 has links to pages 3, 747213, and so on. Note that links are not necessarily two-way. The application should save the IDs of orphan pages in the output. **Orphan pages** are pages that have no incoming links from other Wikipedia pages. This means that they do not appear in the right-hand side after the colon in any line, as they are pages that are not linked to from any other pages in the Wikipedia dataset. **Note:** *A page that links to itself is NOT an orphan page.*

The following is the sample command we will use to run the application:

```
# spark-submit OrphanPagesSpark.py dataset/links/ partC
```

If you want to check your output, run:

```
# cat partC
```

If you want to check a **part** of your output, run:

```
# head partC
```

Here is a part of the output of this application:

```
100005
100019
100027
100028
100055
100072
100105
100110
100162
100219
```

The order of lines matters. Please sort your output (key value) in alphabetic order.

Exercise D: Top Popular Links

In this exercise, you will implement an application to find orphan pages in Wikipedia. We have provided a template for this exercise in the following file: **TopPopularLinksSpark.py**

You need to make the necessary changes to parts marked with **TODO**.

Your application takes a list of Wikipedia links (not Wikipedia titles anymore) as an input. All pages are represented by their ID numbers. Each line starts with a page ID, followed by a list of pages that the ID has a link to. The following is a sample line in the input:

```
2: 3 747213 1664968 1691047 4095634 5535664
```

In this sample, page 2 has links to pages 3, 747213, and so on. Note that links are not necessarily two-way. The application should save the IDs of the top **10** popular pages and the number of links to them in the output. A page is popular if more pages are linked to it. **Use the method in Sorting to select top links.**

The following is the sample command we will use to run the application:

1

spark-submit TopPopularLinksSpark.py dataset/links/ partD

If you want to check your output, run:

1

cat partD

Here is the output of an application that selects the top **5** popular links:

```
1921890 721
481424  729
5302153 1532
84707   1060
88822   1676
```

The order of lines matters. Please sort your output (key value) in alphabetical order. Also, make sure the key-value pair in the final output are tab-separated.

Exercise E: Popularity League

In this exercise, you will implement an application to find the most popular pages in Wikipedia. Again, we have provided a tempalte for this exercise in the following file: **PopularityLeagueSpark.py**

You need to make the necessary changes to parts marked with **TODO**.

Your application takes a list of Wikipedia links as input. All pages are represented by their ID numbers. Each line starts with a page ID, followed by the pages the ID links to. The following is a sample line in the input:

```
2: 3 747213 1664968 1691047 4095634 5535664
```

In this sample, page 2 has links to pages 3, 747213, and so on. Note that links are not necessarily two-way.

The **popularity** of a page is determined by the number of pages in the whole Wikipedia graph that link to that specific page. (Same number as **Exercise D**)

The application also takes a list of page IDs as an input (also called a **league list**). The goal of the application is to calculate the rank of pages in the league using their popularity.

A **page's rank** is the number of pages in the **league** with less popularity than the original page.

The following is the sample command we use to run the application:

1

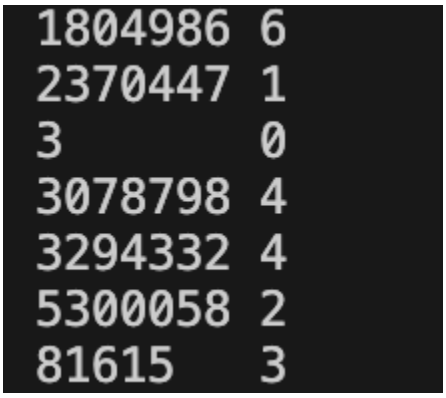
spark-submit PopularityLeagueSpark.py dataset/links/ dataset/league.txt partE

If you want to check your output, run:

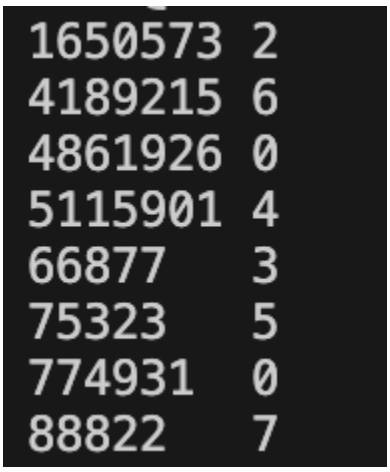
1

cat partE

Here is the output with **League**={5300058,3294332,3078798,1804986,2370447,81615,3,1}):



Here is the output with **League**={88822,774931,4861926,1650573,66877,5115901,75323,4189215}):



The order matters. Please sort your output (key value) in alphabetic order. Also, make sure the key and value pairs in the final output are tab-separated.

Note that we will use a different League file in our autograder runs. **MP5 CONCERNS & FAQs:** 1. Issue: Autograder Feedback - java.io.IOException: Failed to connect to ip-xx-x-xx-xxx.ec2.internal/xx Feedback: Your driver is facing memory issues. Leading to the error stack.

One possible reason is you are using CountByValue in part D which is an action. It returns the count of each unique value in each individual worker as a map to driver program and driver program is responsible to merge it. This merging operation leads to the memory issue. Use reduceByKey instead in reduceByKey what happens is the driver moves your keys around among workers and counting is done on workers. This should help solve your memory issues. Always remember to use countbyvalue only when you have small rdd. Hope this helps. The same issue might arise when you try and use actions (such as reduce)

very early on in your algorithms. Another common scenario that leads to this error is when you collect too early and try to filter data using python rather than spark.

Some useful links: https://www.clairvoyant.ai/blog/apache-spark-out-of-memory-issue https://stackoverflow.com/questions/52915466/when-to-use-countbyvalue-and-when-to-use-map-reducebykey https://data-flair.training/forums/topic/explain-countbyvalue-operation-in-apache-spark-rdd/

How to submit

When you're ready to submit, you can upload files for each part of the assignment on the "My submissions" tab.

Like

Dislike

Report an issue